

Photogrammetrie auf dem Laptop: Möglichkeiten und Grenzen einer in Python implementierten 3D Rekonstruktions-Pipeline

Rudolf Hoffmann¹, Frank Neumann¹

¹HTW Berlin, Fachbereich 2 Informatik in Ingenieurwissenschaften, Wilhelminenhofstr. 75a, 12459 Berlin, Rudolf.Hoffmann@Student.HTW-Berlin.de, www.htw-berlin.de

Abstract: Wie weit trägt eine selbst implementierte Photogrammetrie-Pipeline auf einem handelsüblichen Laptop, ohne GPU-Cluster und ohne kommerzielle Software? Der Beitrag beantwortet diese Frage anhand eines studentischen Implementierungsprojekts und grenzt dabei genau ab, was „selbst implementiert“ bedeutet: Etablierte Bausteine wie SIFT, FLANN-Matching, die robusten Schätzer für Fundamental-, Essential- und Homographiematrix sowie PnP stammen aus OpenCV, der nichtlineare Löser aus SciPy; eigener Code ist die gesamte Rekonstruktionslogik samt Problemformulierung des Bundle Adjustment. Auf einem Datensatz mit 67 Aufnahmen und mitgelieferten Ground-Truth-Posen registriert die Pipeline alle Kameras und liefert eine formtreue Punktwolke aus 39.721 Punkten bei 2,08 px Reprojektions-RMSE. Der Vergleich mit der Ground Truth zeigt jedoch Schwächen, die diese pipeline-eigene Kennzahl nicht anzeigt: Die geschätzte Brennweite liegt 47 % neben dem wahren Wert und die Kameraorientierungen weichen im Median um 6,29° ab. COLMAP erreicht auf denselben Bildern, derselben CPU und mit derselben Merkmalszahl 0,2 % Brennweiten- und 0,11° Orientierungsfehler in einem Viertel der Laufzeit. Die Genauigkeit steckt also in den Daten, und die Lücke liegt in der Umsetzung.

Keywords: Structure-from-Motion; Photogrammetrie; Python; Punktwolke; Bundle Adjustment; Studierendenprojekt

1 Einleitung

Aus einer Handvoll gewöhnlicher Fotos ein dreidimensionales Modell zu berechnen, gehört heute zu den Standardwerkzeugen von Vermessung, Denkmalpflege, Robotik und AR/VR. Die zugrundeliegende Technik ist in allen Fällen *Structure-from-Motion* (SfM), also die gleichzeitige Schätzung von Szenengeometrie und Kamerapositionen aus reinen Bilddaten. In der Praxis greifen die meisten Anwender zu fertigen Werkzeugen wie COLMAP [2] oder Meshroom, die gute Ergebnisse liefern, die zugrundeliegenden Algorithmen aber als Blackbox verbergen.

Das hier beschriebene Projekt geht den umgekehrten Weg und baut die Verarbeitungskette selbst (Fig. 1), mit frei verfügbaren Bibliotheken und ohne spezialisierte Hardware. Daraus ergeben sich zwei Ziele. Das erste ist didaktisch: Jede Verarbeitungsstufe soll durch Zwischenergebnisse sichtbar werden. Das zweite ist evaluativ: Die Ergebnisse werden gegen Ground-Truth-Kameraposen und gegen COLMAP vermessen, um Möglichkeiten und Grenzen einer reinen Python-Umsetzung belastbar einzuordnen. Der Beitrag berichtet beides, einschließlich der Fehler, die dabei sichtbar geworden sind.

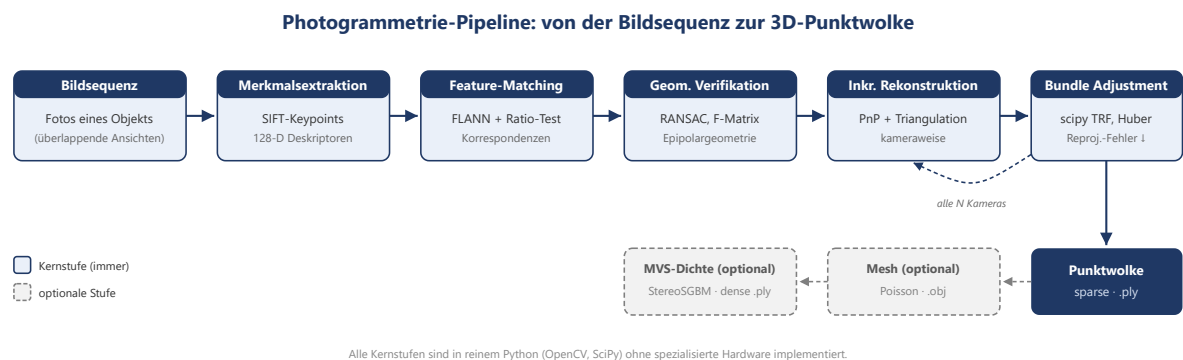


Fig. 1: Verarbeitungskette von den Eingabebildern über Merkmale, Matching, geometrische Verifikation, inkrementelle Rekonstruktion und Bundle Adjustment bis zur Punktwolke und optional zum Mesh.

2 Eigener Code und verwendete Bibliotheken

SfM geht auf die Mehrbildgeometrie der 1980er- und 1990er-Jahre zurück und wurde von Hartley und Zisserman [1] systematisiert. Drei geometrische Grundideen tragen jede Umsetzung. Die **Epipolargeometrie** verbindet zwei Bilder über die Fundamentalmatrix und schränkt den Partner eines Bildpunktes auf eine Gerade ein, was falsche Korrespondenzen geometrisch verworfbar macht. Die **Triangulation** rekonstruiert einen 3D-Punkt als Schnitt zweier Sehstrahlen, umso stabiler, je größer der Winkel zwischen ihnen ist. Das **Bundle Adjustment** (BA) optimiert schließlich alle Kameraposen und 3D-Punkte gemeinsam auf minimalen Reprojektionsfehler.

Die Formulierung „selbst implementiert“ verlangt eine genaue Abgrenzung, denn SIFT [4], FLANN und RANSAC sind etablierte Verfahren, deren Implementierungen niemand ohne Not neu schreibt. Aus den Bibliotheken stammen daher die numerischen Primitive, also einzelne, klar umrissene Rechenschritte. Selbst geschrieben ist alles, was diese Primitive zu einer Rekonstruktion verbindet, sowie das vollständige Fehlermodell des Bundle Adjustment. Tab. 1 trennt beides auf.

Stufe	Aus Bibliotheken	Eigener Python-Code
Merkmale	<code>cv2.SIFT_create</code>	Bildladen mit EXIF-Rotation, Schätzung der Intrinsik, Zwischenspeicherung
Matching	<code>cv2.FlannBasedMatcher</code> , <code>cv2.kmeans</code>	Ratio-Test, Cross-Check, Auswahl der Kandidatenpaare (erschöpfend, sequenziell, Vokabularbaum)
Verifikation	<code>findFundamentalMat</code> (USAC_MAGSAC), <code>findEssentialMat</code> , <code>recoverPose</code> , <code>findHomography</code>	Hartley-Normierung, Planaritätsprüfung nach Torr, Inlier-Buchführung, Zusammenhangskomponenten per Union-Find
Rekonstruktion	<code>triangulatePoints</code> , <code>solvePnP</code> , <code>solvePnPRefineLM</code>	Wahl des Startpaars, Registrierungsreihenfolge, Verwaltung von Tracks und Beobachtungen, Akzeptanzkriterien, Ausreißerentfernung, Retriangulation
Bundle Adjustment	<code>scipy.optimize.least_squares</code> (Trust-Region-Reflective)	Parametrisierung, Residuen inklusive Brown-Conrady-Verzeichnung, dünnbesetzte Jacobi-Struktur, adaptive Huber-Skala, Divergenzschutz, lokales BA-Fenster
Dichte Stufe und Mesh	<code>cv2.StereoSGBM</code> , Open3D (Poisson) [3]	Auswahl der Stereopaare, Sichtbarkeitsfilter, Reinigung, Farbübertragung, PLY-Export

Tab. 1: Aufteilung zwischen Bibliotheksaufrufen und eigenem Code.

Die Antwort auf die Frage, ob wirklich alles Python ist, lautet damit differenziert: Der gesamte Projektcode ist Python, rund 9.700 Zeilen, keine eigene Zeile C++ oder CUDA. Die aufgerufenen Bibliotheken sind ihrerseits in C++ geschrieben, so dass die rechenintensiven Primitive kompiliert ausgeführt werden und Python die Steuerungsschicht bildet. Für die Laufzeit ist das günstiger, als es klingt: Der in Abschnitt 5 genannte Engpass entsteht innerhalb der OpenCV-Aufrufe und nicht im Python-Code darum herum.

3 Die Pipeline

Für jedes Bild werden bis zu 8.000 SIFT-Merkmale detektiert. Sie konzentrieren sich auf texturierte Flächen, während strukturlose Regionen leer bleiben, was später die Grenze der Pipeline bestimmt. Korrespondenzen zwischen Bildpaaren findet eine FLANN-basierte Suche nach den beiden nächsten Nachbarn; der Ratio-Test nach Lowe behält nur Zuordnungen, deren bester Treffer deutlich eindeutiger ist als der zweitbeste. Verglichen werden standardmäßig alle Bildpaare.

Jedes Paar durchläuft anschließend vier Filter. Eine Hartley-Normierung stabilisiert die Pixelkoordinaten, `USAC_MAGSAC` schätzt robust die Fundamentalmatrix, ein Vergleich mit einer konkurrierenden Homographie verwirft nach dem Kriterium von Torr planare und damit degenerierte Paare, und aus der Essential-Matrix folgt über die Cheiralitätsbedingung die relative Kamerapose. Ein Union-Find-Verfahren behält von den Zusammenhangskomponenten des Szenengraphen nur die größte.

Die Rekonstruktion wächst danach kameraweise. Als Startpaar dient das Paar mit dem größten Produkt aus Basislinie und Inlier-Zahl bei einem Triangulationswinkel von mindestens 5° . Iterativ wird jeweils das Bild mit den meisten 2D-3D-Korrespondenzen per PnP mit RANSAC und anschließender Levenberg-Marquardt-Verfeinerung registriert; neue Punkte werden mit allen sichtbaren Kameras trianguliert und nur bei positiver Tiefe, hinreichendem Winkel und kleinem Reprojektionsfehler übernommen. Nach jeweils fünf Kameras und am Ende läuft ein Bundle Adjustment über `scipy.optimize.least_squares` mit selbst aufgebauter dünnbesetzter Jacobi-Struktur, einem Projektionsmodell mit radialer Verzeichnung und einem Huber-Loss, dessen Skala aus der Streuung der Startresiduen abgeleitet wird.

Optional erzeugt die Pipeline über StereoSGBM eine dichte Wolke und über Screened Poisson in Open3D [3] ein Mesh. Als Referenzsystem dient COLMAP [2], aufgerufen über denselben Einstiegspunkt auf identischen Eingabedaten. Verwendet wurde COLMAP 4.1.1 ohne CUDA, so dass beide Systeme ausschließlich auf der CPU desselben Rechners arbeiten.

4 Ergebnisse

Ausgewertet wird der Datensatz „Buddha“ aus dem AliceVision-Projekt: 67 Aufnahmen einer genoppten Statue auf einem Drehteller, 2736×1540 Pixel, mit einer Ground-Truth-Projektionsmatrix je Bild. Beide Systeme erhalten dieselben Bilder, dieselbe Merkmalszahl und erschöpfendes Matching. Vor jedem Fehlermaß richtet eine Sim(3)-Anpassung nach Umeyama die geschätzten Kamerazentren auf die Ground Truth aus, da monokulares SfM skalenfrei ist; Positionsfehler sind auf die Ausdehnung der Ground-Truth-Trajektorie normiert. Tab. 2 stellt beide Systeme gegenüber.

Kennzahl	Eigene Pipeline	COLMAP
Registrierte Kameras	67 von 67	67 von 67
3D-Punkte	39.721	35.538
Mittlere Tracklänge	2,70	4,69
Reprojektions-RMSE	2,08 px	nicht exportiert
Geschätzte Brennweite	2.736,0 px	1.857,5 px
Brennweitenfehler	+47,0 %	-0,2 %
Rotationsfehler, Median und Maximum	6,29° / 20,28°	0,11° / 0,20°
Positionsfehler, Median und Maximum	2,06 % / 11,60 %	0,02 % / 0,05 %
Laufzeit	1.135,5 s	293 s

Tab. 2: Gemessene Kennzahlen gegen Ground Truth, beide Systeme mit 8.000 Merkmalen je Bild auf demselben Rechner. Die Ground-Truth-Brennweite beträgt 1.860,9 px.

Qualitativ liefert die eigene Pipeline auf diesem dicht abgetasteten, gut texturierten Datensatz eine vollständige und formtreue Rekonstruktion: alle 67 Kameras registriert, 39.721 Punkte, Objektform und Farbgebung eindeutig erkennbar, weniger als 1 % Ausreißer. Der Reprojektionsfehler von 2,08 px im RMSE und 0,84 px im Median bestätigt das scheinbar.

Das zentrale Ergebnis steht jedoch in der Zeile zur Brennweite. Die Bilder tragen kein EXIF, weshalb die Intrinsik-Schätzung auf die Heuristik $focal = \max(W, H)$ zurückfällt und 2.736 px ansetzt, also die Bildbreite statt der wahren 1.860,9 px. Das Bundle Adjustment korrigiert diesen Fehler nie: Über alle 67 Kameras protokolliert jede Runde eine Änderung von exakt null. Zwei Erklärungen wurden experimentell ausgeschlossen. Die Schranken des Optimierers liegen bei 1.368 px und 5.472 px und enthalten den wahren Wert; eine Parameterskalierung als Ursache widerlegt ein Kontrolllauf mit aktivierter Jacobi-Skalierung, bei dem sich die Brennweite ebenfalls nicht bewegt. Es bleibt die Erklärung, dass die Rekonstruktion bei der falschen Brennweite bereits in sich konsistent ist: Startpose, Triangulation und PnP wurden alle mit 2.736 px gerechnet, die Struktur ist entsprechend projektiv verformt, und das BA hat nichts mehr zu gewinnen. Dazu passt der Verlauf der Optimierung. Im visualisierten Referenzlauf steigt der Reprojektions-RMSE über vierzehn BA-Runden von 7,55 px bei sieben Kameras auf 10,88 px bei 67, statt zu fallen; die Kurven vor und nach jeder Runde liegen dabei praktisch übereinander.

COLMAP entkräftet auf denselben Bildern jede Ausrede, die man dafür anführen könnte. Es startet ebenfalls ohne Kalibrierung, verfeinert die Brennweite aber bis auf 1.857,5 px und liegt damit 0,2 % neben der Ground Truth. Der Datensatz enthält also genügend Information, um die Brennweite zu bestimmen; die eigene Umsetzung holt sie nur nicht heraus. Entsprechend fallen die Posenfehler aus: 0,11° gegenüber 6,29° im Median. Ein zweiter Unterschied erklärt einen Teil davon. COLMAP verknüpft im Mittel 4,69 Beobachtungen zu einem 3D-Punkt, die eigene Pipeline nur 2,70; längere Tracks binden jede Kamera an mehr gemeinsame Struktur und machen die Lösung steifer. Fig. 2 zeigt beide Wolken nebeneinander.

Eigene Pipeline
39,721 Punkte

COLMAP
35,538 Punkte

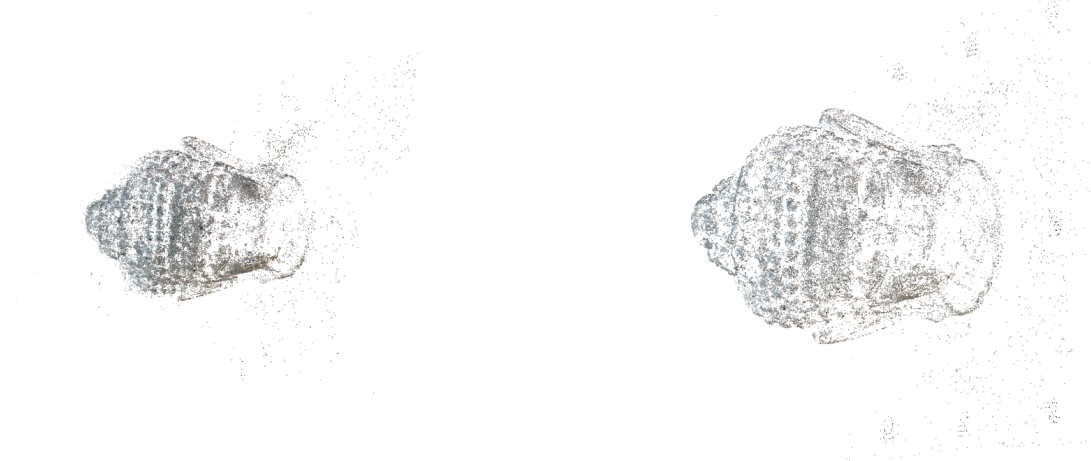


Fig. 2: Dieselben 67 Bilder, links die eigene Pipeline, rechts COLMAP. Beide Wolken sind über eine Sim(3)-Anpassung der Kamerazentren in dasselbe Koordinatensystem gelegt und aus derselben Richtung mit identischem Maßstab gerendert. Die eigene Wolke enthält mehr Punkte, zeichnet die genoppte Oberfläche aber diffuser. Vor allem fällt sie kleiner aus: Bezogen auf die ausgerichteten Kamerazentren beträgt ihr mittlerer Objektradius das 0,69-fache, während der Brennweitenfehler von +47 % genau den Faktor 0,68 vorhersagt. Die projektive Verformung ist damit direkt sichtbar und quantitativ bestätigt.

Ein letzter Befund betrifft die Wiederholbarkeit. Vier byte-identische Aufrufe auf denselben 20 Bildern registrierten 13, 13, 14 und 6 Kameras, eine Streuung von 57 %.

5 Diskussion

Auf dicht abgetasteten, gut texturierten Szenen arbeitet die Pipeline zuverlässig und bleibt mit 1,5 GB Spitzenspeicher im Rahmen eines gewöhnlichen Laptops. Ebenso deutlich sind die Grenzen, und für jede lässt sich die Ursache im Code benennen.

Die Intrinsik ist die gravierendste Schwäche. Ohne EXIF rät die Pipeline `focal = max(W, H)`, das Bundle Adjustment korrigiert den Wert nicht, weil die Rekonstruktion bei dieser Brennweite bereits in sich konsistent ist, und eine Option zur Vorgabe einer bekannten Kalibrierung existiert nicht. Dass es sich um ein Implementierungsproblem und nicht um eine Grenze der Daten handelt, zeigt COLMAP auf denselben Bildern.

Die Ergebnisse sind nicht reproduzierbar. `cv2.setRNGSeed` wird nirgends aufgerufen, so dass `USAC_MAGSAC` und `solvePnP_Ransac` aus dem prozessglobalen Zufallszahlengenerator von OpenCV ziehen. Solange das offen ist, lässt sich der Nutzen jeder weiteren Verbesserung nicht sauber messen.

Die Skalierungsgrenze liegt beim Matching, nicht beim Bundle Adjustment. Erschöpfendes Matching stellt bei 67 Bildern 91 % der Laufzeit, bei praktisch konstanten Kosten je Bildpaar von rund 0,5 s, während das BA mit 38,2 s nicht ins Gewicht fällt. Das widerspricht der Erwartung, der voreingestellte SciPy-Löser werde als Erstes zum Engpass, weil er kein Schur-Komplement nutzt. Der Code-Befund stimmt, die Laufzeitfolge nicht: Das BA wächst zwar schneller als linear, startet aber von einem so kleinen Betrag, dass beide Kurven sich erst weit außerhalb des vermessenen Bereichs schneiden würden.

Die Tracks sind zu kurz und ohne Schleifenschluss driftet die Lösung. Bei einer mittleren Tracklänge von 2,70 stammt die Mehrzahl der Punkte aus nur zwei Bildern, und rund 8 % sind exakte Duplikate nicht verschmolzener

Tracks. Zugleich hängt die Registrierung jede neue Kamera an den bestehenden Verbund an, so dass sich Posenfehler über den Rundgang aufsummieren, was der steigende BA-Fehler direkt zeigt.

Texturarme und planare Szenen bleiben die theoretisch erwartete Grenze. SIFT findet dort zu wenige Merkmale, und bei dominanter Ebene erklärt eine Homographie die Korrespondenzen ebenso gut, weshalb die Verifikation solche Paare verwirft. Ein falsches Ergebnis wird so vermieden, der Bildgraph verliert aber Kanten.

Der Vergleich mit COLMAP fällt damit weniger algorithmisch als praktisch aus. Die Grundstruktur ist in beiden Systemen ähnlich, und beide liefen hier auf derselben CPU mit denselben Bildern. Der Unterschied liegt in der Ausführung, also in belastbaren Startwerten, konsequenter Track-Verwaltung, ausgereifter Ausreißerbehandlung und einem für dieses Problem gebauten Solver.

6 Fazit

Eine vollständig in Python geschriebene SfM-Pipeline rekonstruiert einen gut abgetasteten, texturreichen Datensatz aus 67 Bildern vollständig und formtreu, auf einem handelsüblichen Laptop und mit transparenten Zwischenergebnissen. Das wichtigste Ergebnis des Projekts ist jedoch methodisch: Die Selbstauskunft der Pipeline über ihre eigene Qualität ist unzuverlässig. Ein Reprojektionsfehler von 2,08 px signalisiert eine saubere Rekonstruktion, während die Brennweite 47 % danebenliegt und die Kameraorientierungen im Median um mehr als 6° verdreht sind. COLMAP liefert auf denselben Bildern die Gegenprobe und zeigt mit 0,11° Orientierungsfehler in einem Viertel der Laufzeit, dass die Daten die Genauigkeit hergeben und die Lücke ausschließlich in der eigenen Umsetzung liegt. Für ein Lernprojekt ist das die nützlichste Form eines Ergebnisses, denn sie benennt nicht nur eine Grenze, sondern belegt, dass sie überwindbar ist. Die Reihenfolge der Weiterarbeit folgt daraus unmittelbar: erst den Zufallszahlengenerator setzen, dann eine Kalibrierungsvorgabe und eine Brennweitensuche beim Startpaar, danach die Skalierung des Matchings.

Literatur

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2003.
- [2] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104-4113.
- [3] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," arXiv:1801.09847, 2018.
- [4] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91-110, 2004.